

I. GIỚI THIỆU

1. Trò chơi Nansuke

Nansuke (tên tiếng Nhật: Nanba Sukereton) là một trò chơi logic Nhật Bản được chơi trên một ma trận kích thước $n \times n$ và với danh sách các số cho trước. Người chơi điền vào các ô trống các số cho trước trong danh sách sao cho chúng thỏa mãn luật của trò chơi.

5	1			1
	5	3	1	5
5		1		3
1	3	5	3	
3			5	3

Hình 1.1: Ví dụ về trò chơi Nansuke

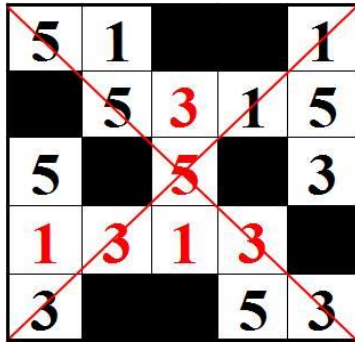
2. Các ràng buộc khi giải trò chơi Nansuke

Trò chơi Nansuke bao gồm 2 luật về chọn số và điền vào ô trống trên hàng và cột :

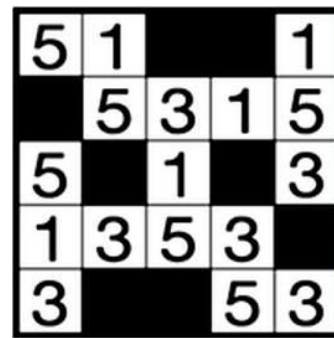
2	3	4
15	51	153
35	53	315
		513

Hình 1.2: Ví dụ đầu vào của Nansuke

- **Luật 1:** Điền vào tất cả các ô trống các số được cho trước trong danh sách.



Hình 1.3: Ví dụ về vi phạm luật 1



Hình 1.4: Cách giải đúng luật 1

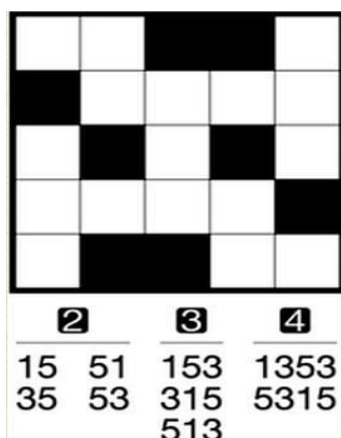
Hình 1.2 là một ví dụ về đầu vào của trò chơi với danh sách các số cho trước.

Trong mỗi hàng tính từ trái sang phải và mỗi cột tính từ trên xuống dưới, các giá trị số được điền vào phải chính xác so với các số cho trước.

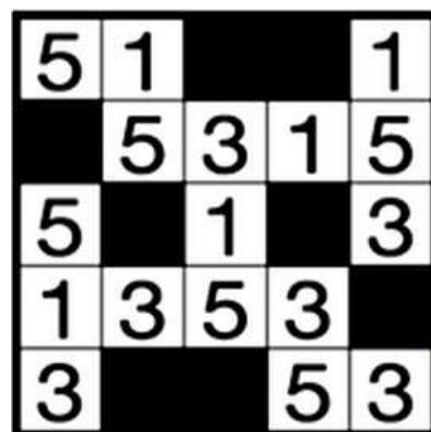
Hình 1.3 cho thấy tại hàng thứ 4 số 1313 và cột thứ 3 số 351 không có trong danh sách các số đầu vào cần phải điền. Điều này đã vi phạm luật 1 của Nansuke. Hình 1.4 trình bày cách giải đúng với đầu vào tương tự hình 1.3.

- **Luật 2:** Các số được điền từ trái sang phải hoặc từ trên xuống dưới. Mỗi ô chỉ được điền chính xác 1 chữ số

Ví dụ trên hình 1.4 cho thấy mỗi một ô trắng trong ma trận sẽ chỉ được điền một chữ số.



Hình 1.5: Đầu vào Nansuke



Hình 1.6: Lời giải hoàn chỉnh Nansuke

Trên đây là hai luật cơ bản của Nansuke, với các luật trên ta có thể đưa ra lời giải chính xác, đầy đủ hai luật Nansuke như hình 1.6 với đầu vào là hình 1.5.

II. PHƯƠNG PHÁP SAT ENCODING

1. Các khái niệm cơ bản SAT Encoding

1.1. Bài toán SAT

Bài toán SAT(satisfiability problem) là một bài toán nhằm kiểm tra tính thỏa mãn (*satisfiability hay unsatisfiability*) của một công thức Logic mệnh đề (giá trị của biến Logic là *true* hoặc *false*). Nếu tìm được bộ giá trị của các biến logic làm cho công thức Logic mệnh đề đúng thì công thức đó được gọi là *satisfiability* (SAT), ngược lại là *unsatisfiability* (UNSAT). Đây là bài toán thuộc lớp NP-đầy đủ (*NP-complete*).

Đầu vào của bài toán SAT là một công thức Logic mệnh đề thường được biểu diễn dưới dạng chuẩn tắc hội (CNF - Conjunctive Normal Form – hội của các tuyển sơ cấp).

$$CNF = TSC1 \wedge TSC2 \wedge TSC3 \wedge \dots$$

Ví dụ về bài toán SAT: Cho một công thức logic mệnh đề F:

$$F = (A \vee B \vee C) \wedge (\neg A \vee B \vee \neg C)$$

Trong đó A, B, C là các biến logic mệnh đề (nhận giá trị *true* hoặc *false*), $(A \vee B \vee C)$ và $(\neg A \vee B \vee \neg C)$ là các tuyển sơ cấp (DNF) hay còn gọi là các mệnh đề (clauses).

Công thức F là SAT vì với bộ giá trị $A = false, B = true, C = false$ thì F cho kết quả *true*.

1.2. SAT Solver

SAT Solver là một công cụ có thể chứng minh một cách tự động công thức Logic mệnh đề là SAT hay UNSAT (giải tự động bài toán SAT).

MiniSat là một dạng mã nguồn mở cho phép các nhà nghiên cứu xây dựng và phát triển mở rộng các SAT solver dựa trên nó. Một số tính năng chính của MiniSat:

- Dễ dàng chỉnh sửa
- Hiệu quả cao
- Được thiết kế để tích hợp.

Yêu cầu đầu vào của MiniSat biểu diễn dưới định dạng DIMACS CNF

p CNF VARIABLES CLAUSES

Trong đó:

- VARIABLES là số lượng biến
- CLAUSES là số lượng mệnh đề (số lượng các tuyến sơ cấp trong một công thức CNF).

Ví dụ: Cho mệnh đề: $F = (x_1 \vee x_3 \vee \neg x_4) \wedge (x_2 \vee \neg x_3)$

CNF file:

p cnf 4 2

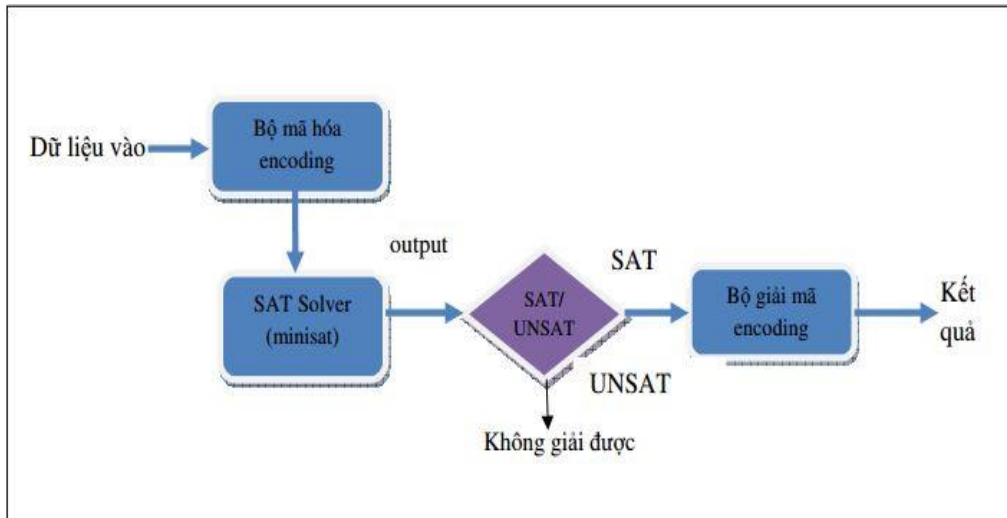
1 3 -4 0

2 -3 0

1.3. SAT Encoding

SAT Encoding là phương pháp biểu diễn vấn đề bằng các công thức Logic mệnh đề và áp dụng SAT Solver vào giải các công thức Logic mệnh đề đó. Từ kết quả trả về của SAT Solver phương pháp đưa ra lời giải của bài toán (khi SAT Solver thông báo công thức là Satisfiable), hoặc thông báo không tìm được lời giải (Unsatisfiable). SAT Encoding đã được áp dụng để giải quyết một số trò chơi Logic: Sudoku, NumberLink, Nansuke, Hitori,...

1.4. Sơ đồ giải bằng SAT Encoding



Hình 2.1 : Sơ đồ giải trò chơi bằng SAT Encoding

Sơ đồ trên gồm các thành phần:

- **Dữ liệu vào:** là bài toán cần giải, ví dụ đầu vào là một ma trận Nansuke, nó gồm danh sách các số cho trước và 1 ma trận đã được bôi đen một số ô.
- **Bộ mã hóa:** Mã hóa các luật chơi của dữ liệu vào thành các mệnh đề CNF theo chuẩn DIMACS để đưa vào SAT Solver. Khi mã hóa sẽ sinh ra file đầu vào rồi đưa tới SAT Solver.
- **SAT Solver:** là công cụ sẽ giải các mệnh đề trong file đầu vào rồi sinh ra file kết quả.
- **Bộ giải mã:** Dịch kết quả trong file kết quả trả về từ SAT Solver, đưa ra lời giải cho bài toán đầu vào.
- **Kết quả:** là đáp án cho dữ liệu đầu vào, nếu đầu vào là một ma trận Sudoku thì kết quả là đáp án của ma trận sudoku đó.

2. SAT Encoding giải Nansuke

Trong phần này sẽ trình bày phương pháp SAT Encoding để giải quyết bài toán Nansuke. Phương pháp này sẽ sử dụng 3 luật sau:

- Luật 1: Mỗi ô trống chỉ được điền 1 chữ số (có giá trị từ 1 đến 9).
- Luật 2: Các số được điền phải đúng giá trị trong danh sách cho trước theo thứ tự từ trái sang phải, từ trên xuống dưới. (luật kéo theo của các chữ số trong 1 số đã cho)

- Luật 3: Các số trong danh sách chỉ được điền duy nhất một lần.

2.1. Mã hóa cho luật 1 của Nansuke

Luật 1 nói rằng: “*Mỗi ô trống chỉ được điền 1 chữ số*”

Tức là các ô trống chỉ được phép điền một chữ số, trong cùng một ô trống không thể tồn tại từ 2 chữ số trở lên. Các số này có giá trị từ 1 đến 9 tùy theo dữ liệu đầu vào.

Gọi $(x_1, x_2, \dots, x_9)$ biểu diễn lần lượt cho sự xuất hiện của các chữ số $(1, 2, \dots, 9)$ trong một ô trống:

Tại vị trí ô trống: nếu ta đã điền x_1 thì sẽ không được điền các số x_2 hoặc x_3 hoặc ... hoặc x_9 nữa:

$$\text{Mệnh đề } x_1 \rightarrow \neg (x_2 \vee x_3 \vee x_4 \vee \dots \vee x_9)$$

Phân tích:

$$x_1 \rightarrow \neg (x_2 \vee x_3 \vee x_4 \vee \dots \vee x_9)$$

$$\equiv \neg x_1 \rightarrow \neg (x_2 \vee x_3 \vee x_4 \vee \dots \vee x_9)$$

$$\equiv \neg x_1 \rightarrow (\neg x_2 \wedge \neg x_3 \wedge \neg x_4 \wedge \dots \wedge \neg x_9)$$

$$\equiv (\neg x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_4) \wedge \dots \wedge (\neg x_1 \vee \neg x_9)$$

Tương tự với việc loại trừ của các chữ số còn lại:

$$x_2 \rightarrow \neg (x_1 \vee x_3 \vee x_4 \vee \dots \vee x_9)$$

$$x_3 \rightarrow \neg (x_1 \vee x_2 \vee x_4 \vee \dots \vee x_9)$$

...

$$x_9 \rightarrow \neg (x_1 \vee x_2 \vee x_3 \vee \dots \vee x_8)$$

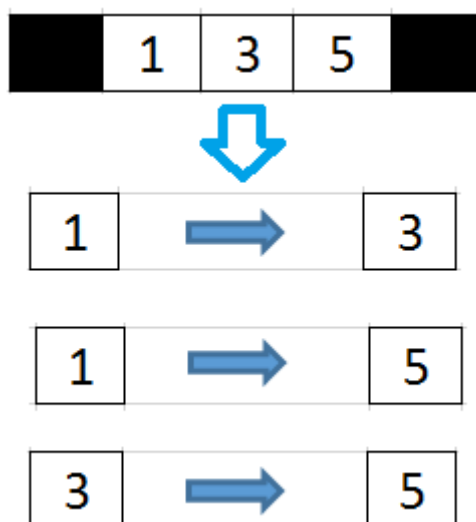
(Nó có thể xuất hiện những cặp số trùng nhau. Ta có thể tính cả hoặc sử dụng giải pháp đề cập ở phần sau)

2.2. Mã hóa cho luật 2 của Nansuke:

Luật 2 nói rằng: “Các số được điền phải đúng giá trị trong danh sách cho trước theo thứ tự từ trái sang phải, từ trên xuống dưới.”

Khi xác định điền số vào ô trống, ta phải điền số đó theo đúng thứ tự xuất hiện các chữ số như dữ liệu đầu vào.

Trong phần này, sẽ lấy một ví dụ điển hình điền 1 số có 3 chữ số. Với các trường hợp các số còn lại thì mệnh đề được áp dụng tương tự.



Hình 2.2 : Sự phụ thuộc các chữ số trong 1 số

Trong hình trên, số 135 được điền vào dãy 3 ô trống liên tiếp theo chiều từ trái qua phải. Từ đó sẽ có 2 sự kéo theo, 1 kéo theo sự xuất hiện 3, 3 kéo theo sự xuất hiện 5. Giả sử đây là hàng 1, của bảng 5x5

$$(x_{121} \rightarrow x_{133}) \wedge (x_{133} \rightarrow x_{145}) \wedge (x_{121} \rightarrow x_{145}) \equiv (\neg x_{121} \vee x_{133}) \wedge (\neg x_{133} \vee x_{145}) \wedge (\neg x_{121} \vee x_{145}) \quad (1)$$

Xét trường hợp loại trừ: chữ số a không kéo theo chữ số b

$$a \rightarrow \neg b \equiv \neg a \vee \neg b \quad (2)$$

Kết hợp (1) và (2): Suy ra mệnh đề logic cho trường hợp 135 điền vào vị trí như hình.

Mở rộng: với n số có 3 chữ số đầu bài cho, áp dụng lần lượt mệnh đề trên cho từng số trên các vị trí có thể điền số có 3 chữ số.

- ⇒ Với n số có m chữ số đầu bài cho, sử dụng luật kéo áp dụng cho từng cặp chữ số trên từng n vị trí có thể điền số có m chữ số.
- ⇒ Sẽ có $n^2 * m * (m - 1)/2$ mệnh đề cho bộ n số có m chữ số.

(Thuật toán code sẽ có phép xác định bộ các vị trí số có mấy chữ số, với luật 3 các số không được sử dụng lặp lại ta suy ra được số vị trí điền số có m chữ số bằng số các số có m chữ số đầu bài cho).

2.3. Mã hóa cho luật 3 của Nansuke:

Luật 3 nói rằng: “*Các số trong danh sách chỉ được điền duy nhất một lần*”

Các số cho trước trong dữ liệu đầu vào nếu đã được điền thì không thể điền lại vào các ô trống khác nữa.

Giả sử ta có số: ab và các cặp vị trí có thể được điền vào: (1, 2) (3, 4), (5, 6): Tức là cặp chữ số (a, b) có thể điền vào tương ứng 1 trong các cặp ô (1, 2), (3, 4), (5, 6), chỉ được điền 1 lần.

Ký hiệu: $a_1 b_2$ tức là (a, b) được điền vào cặp ô (1, 2), cụ thể hơn là a được điền vào ô 1, b được điền vào ô 2. Ta có biểu thức logic cho luật 3 như sau:

$$\begin{aligned}
 & (a_1 \wedge b_2) \rightarrow \neg[(a_3 \wedge b_4) \vee (a_5 \wedge b_6)] \\
 \equiv & \neg(a_1 \wedge b_2) \vee \neg(a_3 \wedge b_4) \vee \neg(a_5 \wedge b_6) \\
 \equiv & \neg a_1 \vee \neg b_2 \vee \neg a_3 \vee \neg b_4 \vee \neg a_5 \vee \neg b_6
 \end{aligned}$$

2.4. Tối ưu dựa trên dữ liệu đầu vào:

2.4.1. Lọc bộ chữ số:

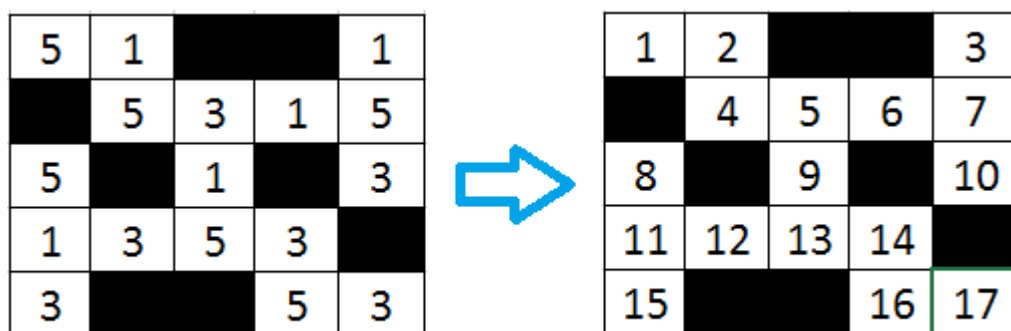
Khi chạy, chương trình sẽ đọc file số các số đầu bài cho, từ đây tạo ra được một mảng A chỉ chứa các chữ số xuất hiện trong các số đầu bài.

Ví dụ:

- mặc định luật 1 sử dụng cả 9 chữ số có thể xuất hiện trên 1 ô trống. Suy ra nếu có n ô trống sẽ cần $n * 9 * 9$ mệnh đề.
- giờ ta có mảng $A = \{1, 3, 5\}$. Luật 1 giờ chỉ sử dụng $n * 3 * 3$ mệnh đề. Chưa kể việc loại trừ trùng lặp.

2.4.2. Đánh số cho ô:

Chương trình sẽ sử dụng một mảng để lưu vị trí các ô trống theo thứ tự từ trái qua phải, trên xuống dưới. Từ đây ngầm hiểu giá trị chỉ số mảng là số thứ tự của ô trống.



Hình 2.3: Đánh số ô trong bảng 5x5

Ta vẫn sử dụng 9 biến để thể hiện việc chữ số nào sẽ xuất hiện ở ô trống, nhưng chỉ khi đưa vào cnf, t mới áp dụng công thức tính vị trí biến $(i + 1) * 9 + a$ với i là vị trí ô trống trong mảng hay số của ô, a là chữ số có thể được điền vào ô trống.

III. THỰC NGHIỆM

Phần này đưa ra kết quả thực nghiệm trên nhiều bảng mẫu với nhiều kích thước như 4x4, 5x5, 9x9 bằng việc sử dụng phương pháp SAT ENCODING trên. Chương trình vẫn có vài vấn đề với các mảng với chiều dài quá 15 ô và tại thời gian báo cáo, chương trình chưa khắc phục kịp những lỗi này. Chúng tôi sẽ khắc phục trong thời gian sớm nhất và cập nhật lại báo cáo.

3.1. Môi trường thực nghiệm:

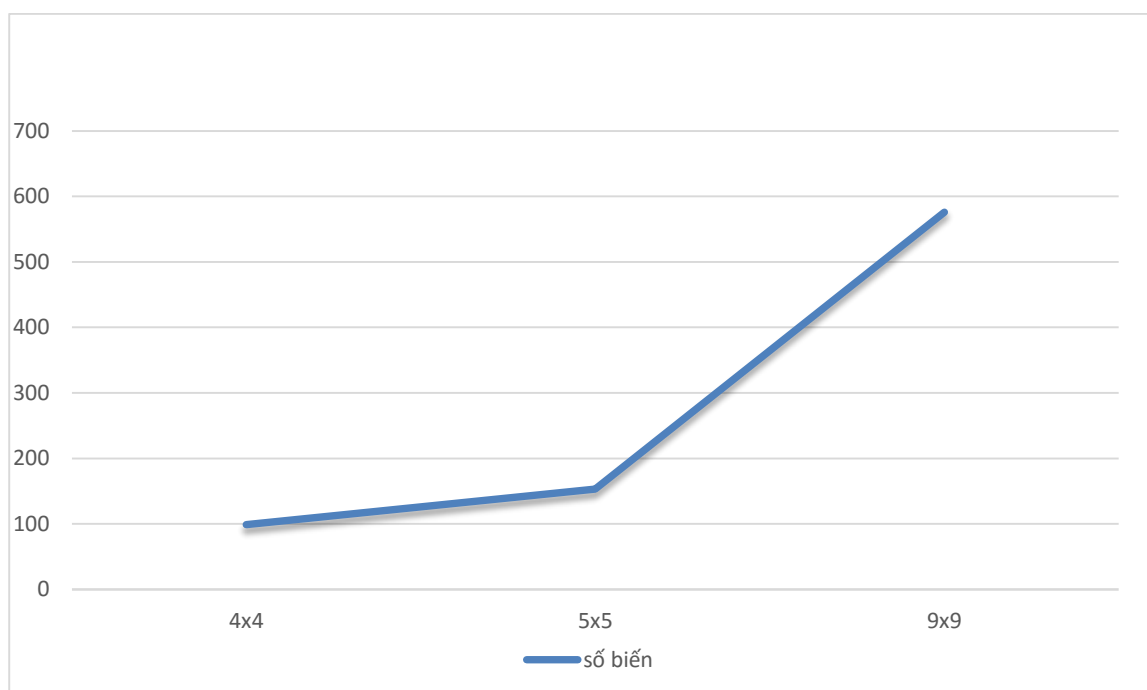
- Cấu hình phần cứng: core i5 2.5GHz, ram 2GB
- Hệ điều hành: Windows 8.1 pro
- Ngôn ngữ lập trình: Java
- SAT solver: MiniSat (sat4j.minisat.SolverFactory)

3.2. Kết quả thực nghiệm:

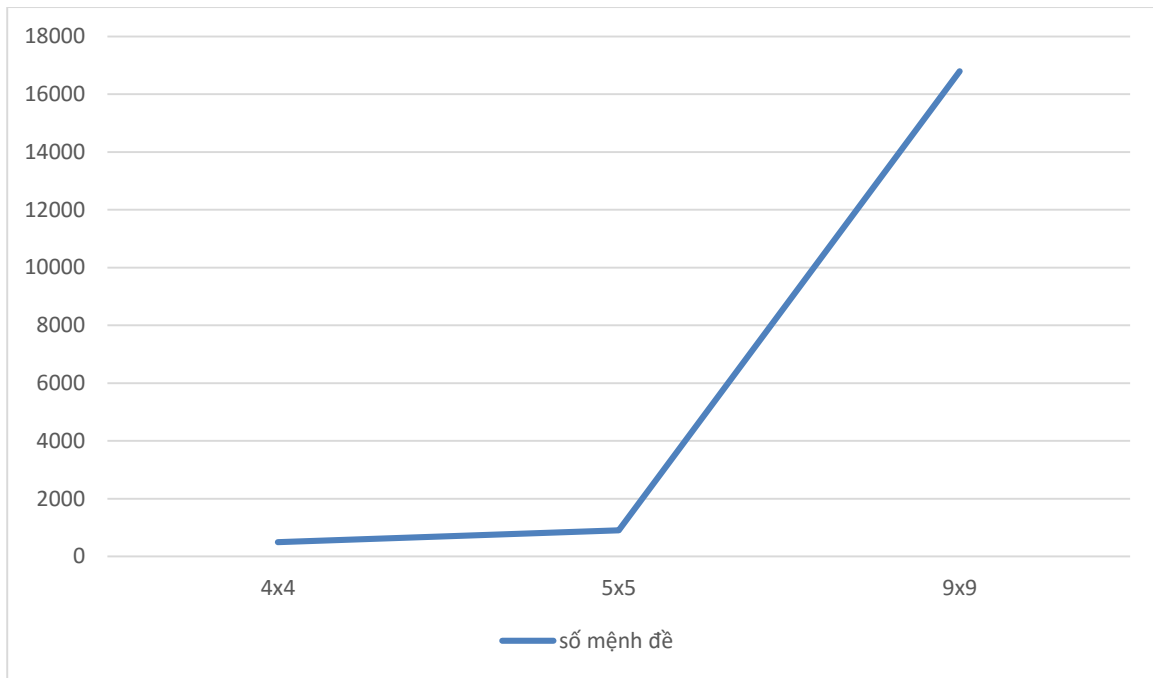
Kích thước bảng	Số biến	Số mệnh đề	Thời gian (ms)
4x4	99	355	100, 90, 90
	90	619	150, 210, 100
	108	495	150, 100, 110
5x5	153	317	270, 90, 100
	162	1411	190, 180, 130
	144	969	200, 110, 117
9x9	576	16794	300, 230, 180

Bảng kết quả thực nghiệm trên các mẫu thử

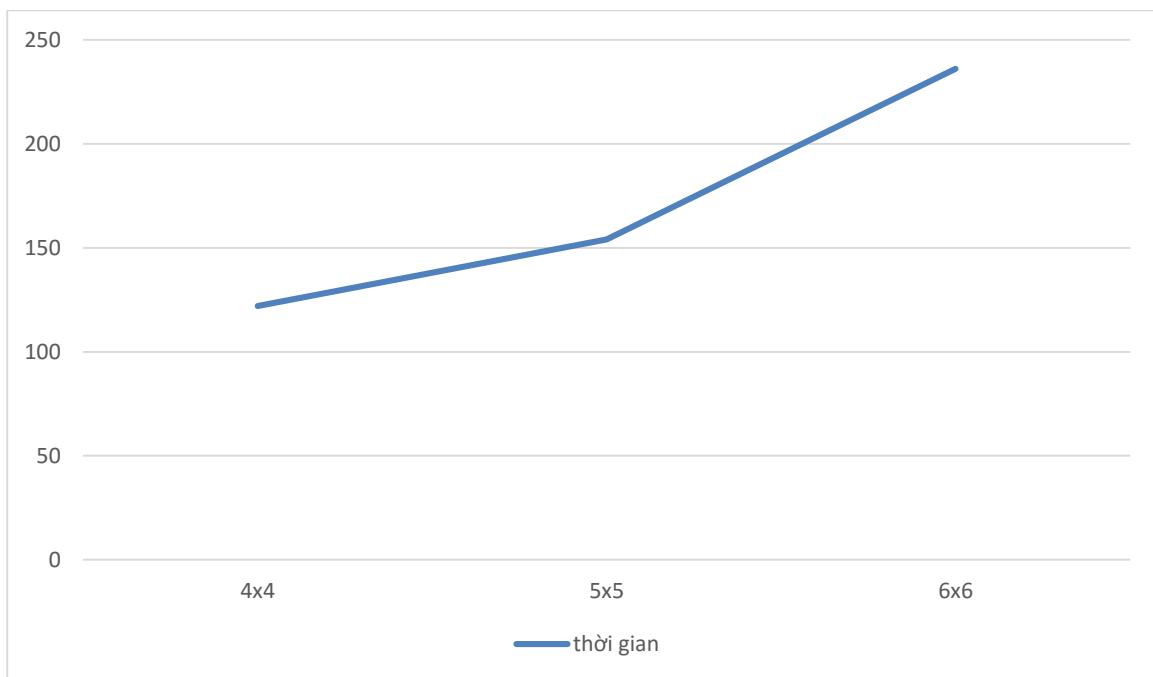
Từ bảng đánh giá trên, chúng tôi tổng hợp ra những biểu đồ sau:



Biểu đồ mối quan hệ giữa số biến và kích thước bảng



Biểu đồ quan hệ giữa số mệnh đề và kích thước bảng



Biểu đồ quan hệ thời gian giải bài toán và kích thước bảng

IV. ĐÁNH GIÁ

Với việc giải quyết bài toán Nansuke sử dụng phương pháp SAT ENCODING và cnf như trên, chúng tôi nhận thấy: theo hướng tăng kích thước bảng

- Số lượng biến tăng dần đều.
- Số lượng mệnh đề tăng phụ thuộc khá nhiều vào bộ số đầu vào.
- Thời gian giải bài toán tuy có tăng nhưng vẫn ở mức chấp nhận được.

Bài toán SAT (Satisfiability) là bài toán chứng minh tính thỏa mãn hay không thỏa mãn của một công thức Logic mệnh đề. Các bài toán đều có khả năng mã hóa theo những công thức logic tương ứng. SAT rất hữu ích trong việc đánh giá, chứng minh các công thức logic hoặc giải các bài toán mang tính chính xác cao.